

Solitito — podsumowanie projektu

System ćwiczenia i rozpoznawania gry na gitarze w czasie rzeczywistym

Aktualizacja: 4 października 2026 — integracja take7, wersja aplikacji 0.5.7

Dokument opisuje wersję 0.5.7 z modelem take7 oraz eksperymenty, które do niej doprowadziły. Dawne pomiary take6 oznaczono jako historyczne; nie są wynikami nowego detektora Rise. Numer modelu jest niezależny od numeru wydania aplikacji.

1. Charakterystyka systemu

Solitito jest trenerem gitarowym działającym w czasie rzeczywistym, stworzonym w Rust. Program pobiera sygnał interfejsu audio lub mikrofonu, rozpoznaje wykonywany materiał i prowadzi użytkownika przez standardy jazzowe, interwały, skale, arpeggia, formuły interwałowe oraz orientację na gryfie.

Rozpoznawanie wykorzystuje jeden ONNX take7 zawierający sieć akordową i osobną gałąź on-setów Rise. Całość przetwarzania — DSP, inferencja oraz interfejs użytkownika — wykonywana jest lokalnie na procesorze, bez połączenia sieciowego i bez usług zewnętrznych.

System udostępnia sześć trybów pracy:

- **Akordy** — pełne standardy jazzowe. Kolor zielony oznacza trafienie dokładne, żółty — triadę lub typowe zastępstwo, czerwony — akord rozpoznany przy sygnale zbyt słabym, by go zatwierdzić.
- **Interwały** — składniki akordu wykonywane pojedynczo, z możliwością wyboru ćwiczonych stopni.
- **Skale** — sekwencyjne przechodzenie dźwięków zgodnie z definicją gamy.
- **Arpeggia** — składniki akordu w sekwencji, na zadanej progresji.
- **Gryf** — losowany jest fragment gryfu obejmujący zestaw strun i cztery progi, po czym zostaje utrzymany; użytkownik proszony jest o kolejne dźwięki leżące w tym obszarze. Tryb służy poznawaniu położenia dźwięków w obrębie jednej pozycji ręki.
- **Formuły** — zbiór interwałów wylosowany nad prymą i grany w dowolnej kolejności, z możliwością postawienia tego samego zbioru na akordzie albo przeniesienia go przez akordy standardu. Tryb opisano w 8.11.

Prace nad projektem rozpoczęto w grudniu 2025 roku. Niniejszy dokument przedstawia architekturę systemu, przebieg prac oraz decyzje projektowe wraz z ich uzasadnieniem.

2. Metodyka prac

W projekcie przyjęto zasadę, że każda zmiana wymaga uzasadnienia pomiarowego, a nie hipotezy. W praktyce oznaczało to opracowanie zestawu **sond** — skryptów odpowiadających na pojedyncze pytanie niewielkim kosztem obliczeniowym, bez konieczności ponownego treningu.

sonda	zagadnienie
<code>verify_annotations.py</code>	czy etykiety opisują zawartość audio?
<code>probe_root.py</code>	jak często pryma wskazana etykietą faktycznie brzmi w oknie?
<code>probe_sources.py</code>	która adnotacja akordowa zbioru GuitarSet jest użyteczna?
<code>probe_quality.py</code>	z jakiego źródła wyprowadzać jakość akordu?
<code>inspect_jams.py</code>	jaka jest rzeczywista zawartość plików JAMS?
<code>latency_material.py</code>	wytwarza szarpnięcia o dokładnie znanych momentach ataku, jako wzorzec
<code>latency_ground_truth.py</code>	wydobywa ataki i wysokości z PRAWDZIWEGO nagrania, do tego samego pomiaru
<code>latency_stats.py</code>	jak późno aplikacja dowiaduje się, co zagrano, i jak często dowiaduje się źle?
<code>latency_rules.py</code>	ile kosztowałyby każda z reguł zaliczania na tym nagraniu?

Pierwsze pięć dotyczy danych i modelu; pozostałe cztery tworzą łańcuch i używane są razem. Materiał jest przygotowywany — albo syntetyzowany, z atakami znanymi z konstrukcji, albo pobierany z nagrania rzeczywistej gry — po czym plik przepuszczany jest przez własną ścieżkę cech aplikacji poleceniem `./solitito --probe plik.wav --step 1`, a powstały wydruk czytają dwa ostatnie skrypty. `latency_stats.py` rozdziela trzy odpowiedzi, jakimi aplikacja dysponuje: głowicę wysokości modelu, estymatę jednoramkową oraz głowicę ataków. `latency_rules.py` odtworza na tym samym wydruku reguły zaliczania i podaje dla każdej, ile przyznałaby zaliczeń, których nikt nie zagrał, oraz ile dźwięków pominęłaby zupełnie. Tabela w 8.11 pochodzi z tego skryptu.

Dwa dalsze narzędzia nie są sondami, lecz należą do tego samego zestawu: `gen_weights.py`, wytwarzający rzadkie jądro CQT wspólne dla trenera i aplikacji, oraz `gp5_to_arpeggio.py`, przekładający plik Guitar Pro na zapis stopniami, który czyta tryb Arpeggia. `hf_cleanup.py` czyści repozytorium punktów kontrolnych przed przebiegiem rozpoczynanym od zera.

Metodyka ta wykazała skuteczność wielokrotnie. Odnotować należy również jej rewers: **hipotezy formułowane przed wykonaniem pomiaru okazywały się błędne w sposób systematyczny**. Zestawienie tych przypadków zawiera rozdział 9.

3. Zbiór syntetyczny

Skrypt `dataset_generator_v2.py` wytwarza w jednym przebiegu plik Guitar Pro **oraz** komplet adnotacji.

3.1. Zawartość

394 bloki o czasie trwania 6 sekund (3 takty przy 120 BPM), obejmujące:

- 12 prym $\times$ {maj, min, maj7, dom7, min7, m7b5, dim7, sus4, aug} w kilku pozycjach na gryfie,
- wszystkie 96 pojedynczych dźwięków (6 strun $\times$ 16 progów).

Struktura bloku: takt pierwszy stanowi atak, drugi — wybrzmienie (tie), trzeci — ciszę. Adnotacja obejmuje przedział `[start + 0,05 s, +3,2 s]`, to jest atak wraz z sustainem, z pominięciem ogona zaniku.

3.2. Weryfikacja akordów

Przed rozpoczęciem generacji skrypt sprawdza **każdy z 21 ruchomych akordów na każdym progu**, weryfikując, czy akord realnie daje deklarowane interwały. Błąd w tabeli akordów zatrzymuje generację zamiast propagować się do zbioru danych.

3.3. Render

Ścieżka gitary jest eksportowana jako sygnał DI i renderowana w środowisku DAW przy użyciu [NAM](#) w dwóch wariantach:

- `synth_dataset_clean.wav` — Fender Deluxe Reverb, brzmienie czyste,
- `synth_dataset_eob.wav` — na granicy przesteru.

Zalecaną częstotliwością próbkowania jest **48 kHz**, z dwóch niezależnych powodów. NAM pracuje natywnie w 48 kHz, wobec czego przy 44,1 kHz wtyczka wykonuje resampling wewnętrzny. Ponadto decymacja do 16 kHz, na których pracuje model, jest przy 48 kHz dokładna ($48000/16000 = 3$), a przy 44,1 kHz — nie ($2,75625$).

3.4. Kalibracja i weryfikacja

Tryb `--calibrate <wav>` wyznacza moment pierwszego ataku na wypadek, gdyby środowisko DAW dodało ciszę na początku pliku. Obsługiwane są formaty PCM 16/24/32-bit oraz zmienneprzecinkowe 32/64-bit, mono i stereo — przy użyciu własnego czytnika WAV, bez zależności zewnętrznych.

Skrypt `verify_annotations.py` porównuje etykietę z **faktyczną zawartością audio**: dominującą klasą wysokości w oknie zestawia się z prymą wskazaną etykietą. Jedyną zależnością jest numpy.

Wartości odniesienia uzyskane z renderu v2:

	clean	eob	wartość losowa
top1	87%	77%	~8%
top3	100%	98%	~25%

Miarodajna jest wartość `top3`, ponieważ w akordzie pryma bywa cichsza od tercji lub kwinty. Wartość `top3 > 75%` kwalifikuje zbiór do treningu; `top3 ≈ 25%` oznacza, że etykiety nie opisują sygnału.

Skrypt weryfikuje ponadto okres bloków wyznaczony z obwiedni energii oraz przesunięcie pomiędzy początkiem adnotacji a atakiem. Oba testy są **komplementarne**: przesunięcie adnotacji o całkowitą wielokrotność bloku trafia w atak sąsiedniego bloku i pozostaje niewykrywalne dla testu czasowego. Wykrywa je dopiero porównanie etykiet z zawartością sygnału.

Przyjęto zasadę nadrzędną: **generator wypisuje etykiety wprost, a niezależny skrypt weryfikuje ich zgodność z audio**. Żaden krok przetwarzania nie odtwarza etykiet z sygnału.

4. Zbiór GuitarSet

GuitarSet obejmuje 360 nagrań z adnotacjami w formacie JAMS, zarejestrowanych przetwornikiem heksafonicznym. Jest to jedyne źródło materiału z rzeczywistego instrumentu wykorzystane w projekcie. Doprowadzenie go do postaci użytecznej wymagało czterech przebiegów treningowych.

Poniżej opisano cztery właściwości zbioru, których pominięcie obniża dokładność modelu.

Dla Rise zarówno nagrania solo, jak i comp dostarczają etykiet początków nut. Adnotacje rozdzielające struny służą tylko jako cele treningu: model słyszy zwykłe audio mono, a aplikacja nie wymaga przetwornika heksafonicznego. Opisane poniżej problemy etykiet akordowych nie uzasadniają odrzucania adnotacji nut solowych.

4.1. Połowa zbioru nie zawiera akordów

Każdy fragment zarejestrowano dwukrotnie: jako `_comp` (akompaniament) oraz `_solo` (improwizacja jednogłosowa). **Adnotacja akordowa jest w obu przypadkach identyczna** — opisuje progresję, nad którą wykonawca improwizował.

Trening głowic akordowych na plikach solo sprowadza się do uczenia modelu, że pojedynczy dźwięk stanowi pełny akord jazzowy. Materiał solowy obejmuje 180 z 360 plików.

Odfiltrowanie tych plików przesunęło wskaźnik `Exact` z **44,8% na 82,3%** w jednym przebiegu.

Cele **pitch** pochodzące z plików solo pozostają w pełni poprawne — jest to rzeczywiste wykonanie jednogłosowe z dokładnymi adnotacjami nutowymi, a więc materiał odpowiedni dla detektora pojedynczych dźwięków. Trener zachowuje zatem stratę pitch na oknach solowych i maskuje wyłącznie prymę oraz jakość (`GUITARSET_SOLO_MODE = "mask_chord"`).

Odnosić należy konsekwencję pośrednią. Sonda `probe_root.py` wyznaczała początkowo słyszalność prymy na wszystkich 360 plikach, uzyskując sufit 64,1%. Na tej podstawie sformułowano wniosek, że wykonawcy jazzowi stosują voicingi bez prymy, a nazwa akordu nie jest funkcją sygnału. Wniosek ten okazał się nieprawidłowy: po odfiltrowaniu materiału solowego pryma brzmi w **97%** okien akompaniamentu.

4.2. Adnotacje akordowe występują w dwóch wariantach

Każdy plik zawiera adnotację `instructed` (akord wynikający z zapisu) oraz `performed` (transkrypcja wykonania). Liczba segmentów jest identyczna, rozkład etykiet — różny:

jakość	instructed	performed	różnica
maj	2640	2106	-534
min	960	460	-500
min7	0	360	+360
maj7	0	430	+430
dom7	480	694	+214
m7b5	240	134	-106
sus	0	132	+132

jakość	instructed	performed	różnica
razem	4320	4320	0

Suma segmentów pozostaje niezmienną, wobec czego nie jest to wybór pomiędzy większą a mniejszą liczbą danych, lecz **przeetykietowanie tych samych nagrań**.

Wiersze `min` oraz `min7` należy rozpatrywać łącznie: **pięćset segmentów oznaczonych w zapisie jako `m` zostało wykonanych jako `m7`**. Trening na adnotacji `instructed` uczy model, aby voicing zawierający septymę małą klasyfikować jako zwykły akord molowy. Błąd ten był następnie obserwowany w aplikacji, gdzie akord `Gm7` rozpoznawano jako `Gm`.

Ponadto adnotacja `instructed` nie zawiera **ani jednego** wystąpienia klas `maj7` i `min7`. Do momentu przełączenia obie klasy pochodziły wyłącznie z dwóch renderów zbioru syntetycznego, to jest z jednego instrumentu przetworzonego przez jeden wzmacniacz. Skutkowało to wartością 100% na zbiorze walidacyjnym (ten sam instrument po obu stronach podziału) przy jednoczesnym braku odporności na materiał rzeczywisty.

Przełączenie na adnotację `performed` przesunęło wskaźnik `Exact` z **82,3% na 92,4%**. Dokładność rozpoznawania prymy pozostała bez zmian: obie adnotacje różnią się co do prymy w **0 z 43 056** porównań.

4.3. Podział zbioru musi przebiegać po źródle

Losowe potasowanie listy segmentów akordowych i podział w proporcji 94/6 umieszcza sąsiadujące takty **tego samego nagrania** po obu stronach podziału — przy identycznym instrumencie, pomieszczeniu, mikrofonie i ujęciu, często przy tym samym akordzie występującym takt później.

W zbiorze syntetycznym zależność jest silniejsza: rendery `clean` i `eob` jednego bloku stanowią to samo wykonanie przetworzone przez inny wzmacniacz, a trafiały do zbioru treningowego i walidacyjnego niezależnie.

Zastosowane rozwiązanie polega na grupowaniu po źródle: całym pliku dla zbioru `GuitarSet` oraz całym bloku (obu renderach) dla zbioru syntetycznego.

Wszystkie wskaźniki walidacyjne ulegają po tej zmianie obniżeniu. Nie stanowi to regresji modelu, lecz usunięcie zawyżenia, które unieważniało wcześniejsze wnioski dotyczące generalizacji. Wartość `root_acc = 98%` uzyskana w przebiegu `take1` była w znacznej mierze artefaktem: przy adnotacjach `both` ten sam segment występował dwukrotnie, wobec czego identyczne okno trafiało zarówno do zbioru treningowego, jak i walidacyjnego.

4.4. Cele pitch wyznaczane z `note_midi`

Adnotacja akordowa opisuje **zamierzoną harmonię** w skali wielu sekund. Okno treningowe obejmuje 0,77 s i często nie zawiera etykietowanej septymy. Model był zatem karany za nieprzewidzenie dźwięku nieobecnego w sygnale — `recall` septym na zbiorze `GuitarSet` wynosił 32%.

Przetwornik heksafoniczny dostarcza adnotacji `note_midi`, opisujących rzeczywiste wykonanie na każdej strunie osobno. Wyznaczenie celów pitch na ich podstawie podniosło `recall` septym z **32% do 96%**.

Przyjęty próg: dźwięk musi brzmieć przez co najmniej 25% okna (`NOTE_MIN_COVER`), aby zostać uwzględniony w celu.

5. Architektura

5.1. Dwa tory sygnału w jednym modelu

Take7 to jeden samowystarczalny `best_model_v2_take7.onnx`, z dwoma niezależnymi wejściami i czterema wyjściami. Przy starcie aplikacja wydziela potrzebną gałąź do pamięci każdego wątku. Nie zapisuje modeli pochodnych. Sam wybór jednego wyjścia pełnego grafu nie pomijał drugiej gałęzi w pomiarze profilera; wydzielenie w pamięci usuwa obliczenia akordowe z cyklu onsetów wynoszącego 16 ms.

```
sygnał mono z gitary
├─ DSP akordów: 16 kHz → FFT 8192 → rzadkie CQT/chroma/bas
│   └─ features [1,48,168] → pryma, jakość, brzmiające nuty
└─ DSP onsetów: 16 kHz → widma Hanna 1024/2048
    └─ short_features [1,770,35] → zdarzenia Rise dla nut
```

Tor akordowy zachowuje cechy take6: 144 log-normalizowane biny CQT, 12 wartości chromy i 12 wartości energii basowej. Rzadkie jądro ma 24 biny na oktawę, obejmując sześć oktaw od C1. Kontekst to 48 ramek przy skoku 256 próbek, około 0,77 s; inferencja wykonywana jest co 40 ms.

Tor onsetów korzysta z okien widmowych 64 i 128 ms, częstotliwości do 4 kHz oraz kompresji amplitudy `log1p`, bez normalizacji całego nagrania. Cechy są zaokrąglane do precyzji `float16` używanej w cache treningowym. Przyczynowy resampling, dopełnienie z lewej i znaczniki czasu odpowiadają trenerowi. Inferencja działa co 256 próbek, czyli 16 ms, na 34 poprzednich ramkach cech i ramce bieżącej. Historia jest utrzymywana ciągle; przyszłe audio nie jest potrzebne. Cykl obliczeń nie oznacza opóźnienia wykrycia wynoszącego 16 ms.

5.2. Baza akordowa i sieć Rise

Gałąź akordowa zachowuje CNN z Squeeze-and-Excitation oraz czterowarstwowy Transformer z take6. Token CLS dostarcza predykcji prymy, jakości i brzmiających klas wysokości. Poprzednia głowica `fc_onset` jest usuwana.

Rise wykorzystuje projekcję surowych cech 770→96 oraz drugą projekcję dodatniej zmiany widma, po których następują cztery przyczynowe konwolucje rezydualne z dylatacjami 1, 2, 4 i 8 oraz 12 logitów wyjściowych. Dodatkowe wejście to, w przestrzeni amplitud, dodatnia różnica między bieżącym widmem a średnią z czterech poprzednich ramek. Bieżąca ramka nie wchodzi do tego tła. Różnica jest ponownie kompresowana do skali cech wewnątrz grafu ONNX.

Wejście/wyjście	Kształt podczas pracy	Znaczenie
<code>features</code>	<code>[1,48,168]</code>	kontekst CQT akordów
<code>short_features</code>	<code>[1,770,35]</code>	przyczynowy kontekst onsetów
<code>root_logits</code>	<code>[1,13]</code>	12 prym i szum
<code>quality_logits</code>	<code>[1,11]</code>	rodzina akordu, pojedyncza nuta lub szum
<code>pitch_logits</code>	<code>[1,12]</code>	brzmiające klasy wysokości, po sigmoidzie
<code>onset_logits</code>	<code>[1,12,35]</code>	prawdopodobieństwa ataku po sigmoidzie; używana ostatnia ramka

Metadane zapisują próg onsetów, opis cech, długość historii i hashe modeli źródłowych. Aplikacja odrzuca niezgodny kontrakt. Prog onsetów jest niezależny od progu brzęcych dźwięków dostępnego w interfejsie.

5.3. Dlaczego zastąpiono poprzednią głowicę onsetów?

Zadaniem jest rozpoznanie świeżego uderzenia konkretnej klasy wysokości, podczas gdy inne dźwięki mogą nadal wybrzmiewać. Trzymana nuta może pasować do interwału żądanego w kolejnym akordzie, mimo że nie została zagrana ponownie. Ogólny atak głośności nie mówi, który dźwięk został właśnie uderzony.

Stara głowica już analizowała przyrost CQT/chromy i zmiany tokenów enkodera. Jej wejście pochodziło jednak z okna FFT o długości 512 ms w torze akordowym, a wykonanie było związane z inferencją akordów. Rise otrzymuje krótsze okna widmowe, uczy się zarówno z widma, jak i jego świeżego przyrostu, i działa niezależnie. Obsługuje jednoczesne ataki kilku klas wysokości bez wymagania ciszy między nimi.

W ćwiczeniach na gitarze użytkownik zgłaszał znaczne ograniczenie zaliczeń przenoszonych między akordami po wprowadzeniu Rise. Wcześniejsze iteracje wymagały też powtarzania części cicho granych nut. Obecna integracja otrzymała pozytywną ocenę z ćwiczeń. Są to obserwacje użytkowe, odrębne od kontrolowanych pomiarów skuteczności; historycznego F1 onsetów z rozdziału 7 nie można porównywać wprost z metrykami zdarzeń Rise. Samo połączenie gałęzi w jeden plik nie poprawia dokładności.

5.4. Podział zadań

Pryma i jakość identyfikują akord; pitch opisuje to, co brzmi. Rise dostarcza nowych ataków do ćwiczeń nutowych przy włączonym **Zaliczaj tylko to, co uderzone**. Sędzia zużywa zdarzenia ze znacznikami czasu, a kolejność i granie pojedynczo nadal określają sposób przechodzenia ćwiczenia. Po wyłączeniu tej opcji pozostają dostępne dotychczasowe reguły brzęcych nut. Akordy mają osobną blokadę.

Głowica jakości pozostaje potrzebna: we wcześniejszym porównaniu na jednym checkpointie osiągnęła 80,5% dokładności, wobec 66,0% dla szablonów opartych na przewidywanych nutach i 59,2% dla szablonów opartych na rzeczywistym zbiorze nut. To historyczne pomiary głowicy akordowej, a nie metryki Rise.

6. Trening

6.1. Trening i wznowianie take7

`dist/model_trainer.py` to wspierany samodzielny skrypt Kaggle, generowany z modułów części akordowej i onsetów. Domyślne `auto` wznowia własny przebieg, a w przeciwnym razie wykorzystuje checkpoint akordowy take6 i trenuje tylko Rise. Gdy nie ma bazy, trenuje również część akordową. `onset_only` wymaga przygotowanej bazy; `full` pomija take6, ale może wznowić własny przebieg. Nowy tag przebiegu rozpoczyna oddzielny eksperyment. `USE_HF=False` pozwala trenować bez historii Hugging Face.

Etap	Obecna rola
Fazy akordowe 1–2	Trening zasadniczy i wybór progu pitch, gdy potrzebna jest baza
Faza akordowa 3	Opcjonalne dostrajanie przy zamrożonym enkoderze; domyślnie wyłączone
Etap onsetów	Trening osobnej gałęzi Rise przy zamrożonych wagach akordowych
Eksport	Scalenie wybranych gałęzi do jednego ONNX i kontrola zgodności wyjść

Snapshoty akordowe take6 nie zawierają wag Rise. Rise zaczyna więc od nowych wag, chyba że podano checkpoint PyTorch w `INITIAL_ONSET`; zapisany stan take7 do wznowienia ma pierwszeństwo. Stara `fc_onset` nie jest wykorzystywana. Optimizer onsetów nigdy nie otrzymuje parametrów części akordowej.

Domyślny przebieg onsetów ma 12 epok. Snapshot ostatniej epoki zapisuje razem model, optimizer, stan generatorów losowych, historię i najlepszy checkpoint. Przy wznowianiu muszą być zgodne tożsamości źródeł, podziały, hashe audio/cech i adnotacje. Niezgodność zatrzymuje przebieg, zamiast po cichu zaczynać od nowa. Samo odtworzenie ścieżki pliku nie pozwala podmienić danych treningowych.

Dla ukończonego wcześniejszego przebiegu z dwoma plikami `MODE="export_only"` łączy ONNX akordów, ONNX onsetów i zapisany próg bez treningu, GPU i ekstrakcji cech. Wynikiem jest `best_model_v2_take7.onnx`; `_chords.onnx` to plik pośredni, nie kompletny model aplikacji. Należy zachować raport treningu i checkpointy. Eksporter sprawdza obie gałęzie względem oryginalnych wyjść ONNX dla trzech kształtów batch/kontekst, zanim zatwierdzi połączony model.

Poniższe uwagi wyjaśniają dwie decyzje zachowane z rozwoju modelu akordowego.

Faza 2 skanowała progi w zakresie 0,30–0,70, optymalizując wskaźnik `exact`. Wskaźnik ten stanowi koniunkcję `argmax(root)` oraz `argmax(quality)`, wobec czego przyjmował identyczną wartość dla wszystkich 41 progów, a wybór był losowy. Obecnie sortowanie odbywa się po F1 głowicy pitch, na którą próg faktycznie oddziałuje.

Fazę 3 wyłączono po zmierzeniu jej efektów w trzech kolejnych przebiegach:

```
take2, 40 epok: pitch_f1 0,9318 → 0,9326 (+0,0008), exact 0,5455 → 0,5445
take3, 4 epoki: F1 0,933 → 0,931, exact 54,6% bez zmian
```

Enkoder pozostaje zamrożony, uczeniu podlegają wyłącznie głowice przy współczynniku uczenia $1e-5$. Próby nie przyniosły użytecznej poprawy, a koszt fazy wynosił około 1,5 godziny obliczeń.

6.2. Straty i maskowanie części akordowej

- **root** — CrossEntropy z wygładzaniem etykiet 0,05,
- **quality** — CrossEntropy z wygładzaniem, sampler ważony po klasie,
- **pitch** — Focal BCE ($\gamma = 2,0$, `pos_weight` 2,5), waga pomocnicza 0,7.

Zastosowano dwa mechanizmy maskowania, oba uzasadnione pomiarem.

`MASK_ROOT_WHEN_SILENT` — strata prymy wyznaczana wyłącznie na oknach, w których pryma faktycznie brzmi. Trening prymy na oknach jej pozbawionych nie prowadzi do wyuczenia percepcji, lecz do zapamiętania progresji zbioru GuitarSet, przy czym wspólny enkoder otrzymuje gradient sprzeczny z celem pitch.

`GUITARSET_SOLO_MODE = "mask_chord"` — pryma i jakość nie otrzymują gradientu z nagrań solowych; głowica pitch otrzymuje go bez zmian.

6.3. Augmentacja części akordowej

- **przesunięcie wysokości** o $\pm N$ półtonów. Istotny szczegół implementacyjny: CQT oraz energia basowa przesuwane są **z wypełnieniem zerami**, chroma — **cyklicznie**. Chroma jest z definicji okrężna, CQT nie jest; zawinięcie pasma basowego na górę zakresu wprowadzałoby dźwięki nieobecne w sygnale.
- **maskowanie czasu i częstotliwości** (SpecAugment),
- **nachylenie widma oraz szum** — symulacja zróżnicowanych torów sygnału.

6.4. Bramka energetyczna części akordowej

Parametr `ENERGY_KEEP_FRAC = 0,55` odrzuca okna o energii poniżej 55% wartości szczytowej segmentu. Uzasadnienie: w fazie zaniku septymy, będąca najcichszym składnikiem voicingu, zanika jako pierwsza, podczas gdy etykieta pozostaje niezmienną. Brak bramki prowadziłby do systematycznego uczenia kolapsu `m7` $\rightarrow$ `m`.

6.5. Metryki części akordowej

Metryki akordowe wyznaczone są **wyłącznie na oknach, w których etykieta opisuje sygnał**, z pominięciem okien solowych. Raportowane są dodatkowo w rozbiciu na okna ze słyszalną prymą i bez niej, ponieważ wartość łączna miesza dwie odmienne populacje.

Wybór najlepszego checkpointu odbywa się według wskaźnika `composite = (root_audible + qual + exact) / 3`. Zastosowanie łącznego `root_acc` premiowałoby model skutecznie odtwarzający progresję zamiast modelu poprawnie analizującego sygnał.

Kontrola diagnostyczna `TRAIN`, wykonywana co 5 epok, wyznacza metryki na danych treningowych bez augmentacji. Odpowiada na pytanie, czy model jest w stanie odwzorować własne dane treningowe. Odpowiedź negatywna wskazuje na cechy lub etykiety jako źródło ograniczenia, nie na generalizację, i oznacza, że zwiększanie liczby epok jest bezcelowe.

6.6. Dane, cele i ocena Rise

Trening onsetów wykorzystuje syntetyczne szarpnięcia i zdarzenia nutowe GuitarSet zarówno z nagrań solo, jak i comp. Adnotacje JAMS rozdzielające struny dostarczają etykiet; wejściem sieci jest zwykłe audio mono, a nie sześć kanałów przetwornika. Pipeline wybiera dostępny wariant mic albo mono mix i zapisuje ten wybór. Początki nut GuitarSet opisują zagrane dźwięki, nie niezależnie zweryfikowaną technikę szarpania.

Wykonawcy GuitarSet 00–03 tworzą trening, 04 walidację, a 05 test. Grupy syntetycznych pobudeń są rozdzielone między te trzy zbiory. Osiem przypadków obejmuje trzymaną prymę, dodatkową tercję/kwintę/oktawę, ponowne szarpnięcia oraz trzymane i powtarzane trójdźwięki. Dostrojony generator `onset-ks-v2` wykorzystuje opóźnienie ułamkowe, aby wysokość renderu zgadzała się z etykietą; sama deterministyczność adnotacji nie zapewniłaby tego.

Domyślna procedura używa BCE z wagą pozytywnych przykładów 4 oraz augmentacji głośności ± 6 dB. Cel oznacza onset klasy wysokości przez 96 ms. Jednoczesne początki tej samej klasy wysokości pozostają ograniczeniem reprezentacji 12-klasowej; raport zawiera liczby takich nałożeń. Wcześniejsze eksperymenty ze stratami paired/ringing pozostają w źródłach, ale take7 ich nie włącza.

Checkpoint jest wybierany metrykami zdarzeń na walidacji, a końcowy przegląd progów korzysta z predykcji wyeksportowanego ONNX. Precision, recall, F1, nadmiarowe zdarzenia na minutę i opóźnienie względem audio są raportowane osobno dla domen i przypadków. Zbiór testowy jest oceniany przy progu wybranym na walidacji. Wynik zdarzeń detektora nie jest wynikiem zaliczania aplikacji: sędzia, bramka i harmonogram UI są odrębnymi elementami. Wcześniej przeanalizowane nagrania gitarowe służą regresji, nie stanowią nieznanego zbioru testowego.

Polecenia i artefakty opisują [instrukcja treningu i eksportu](#) oraz [indeks narzędzi](#).

7. Wyniki i weryfikacja

Historyczny model `v2_take6`, walidacja z podziałem po źródle, z pominięciem okien solowych:

metryka	wartość
dokładność prymy	98,1%
pitch F1	0,909
trafienie dokładne (pryma i jakość)	92,4%
F1 ataków take6 (historyczne)	0,812

Trzy pierwsze wielkości są identyczne w pliku trójgłowicowym i czterogłowicowym: głowica ataków trenowała się przy zamrożonej reszcie sieci. Czwarta podana jest przy progu maksymalizującym F1 na zbiorze walidacyjnym.

Dokładność w podziale na jakości przy najlepszym checkpointcie: `dom7` 97%, `min7` 93%, `min` 92%, `sus` 91%, `maj` 89%, `maj7` 89%; klasy `m7b5`, `dim7` oraz `aug` powyżej 97%.

7.1. Przebieg prac

przebieg	zmiana	Exact
take1–take3	różne, przy podziale z przeciekiem	nieporównywalne
take4	podział po źródle — punkt odniesienia	44,8%
take5	maskowanie nagrań solowych	82,3%
take6	adnotacje <code>performed</code>	92,4%

7.2. Ograniczenie dokładności

Różnica pomiędzy zbiorem treningowym a walidacyjnym w zakresie jakości wynosi **6,5 punktu procentowego** (99,2% wobec 92,7%). Model odwzorowuje dane treningowe. Odpowiada to profilowi ograniczenia przez **generalizację**, nie przez pojemność architektury.

Wniosek praktyczny: zwiększenie liczby epok ani rozmiaru modelu nie przyniesie poprawy. Poprawę przyniesie zwiększenie ilości zróżnicowanego materiału z rzeczywistego instrumentu.

7.3. Kontrola integracji take7

Integrację jednego pliku sprawdzono na kontrolnym połączeniu istniejących wag akordowych take6 i istniejących wag Rise. Wyjścia akordowe/pitch były identyczne z modelem źródłowym dla czterech okien wejściowych. Na pełnym nagraniu AtoA 5682 ramki i 50 wykrytych zdarzeń były zgodne z osobnym modelem Rise, z zerową różnicą prawdopodobieństw, zarówno bez dodatkowego potwierdzania słabszych odpowiedzi, jak i z nim.

Profiler wykazał 32 wykonywane węzły w sesji onsetów oraz 437 w sesji akordów, bez węzłów drugiej gałęzi. Testy runtime objęły również zużywanie zdarzeń, granice ćwiczeń, restarty wejścia i zaliczanie wielodźwięków. Kontrole te potwierdzają zachowanie wyników gałęzi i ich harmonogramu; nie są nową oceną dokładności później wytrenowanych wag take7. Obecny plik take7 przechodzi kontrolę inferencji `--check` w aplikacji.

Nowe metryki treningowe należą do raportu danego przebiegu `training_summary_v2_take7.json`, wraz z wybranym checkpointem, progiem i hashami modeli. Dawne F1 onsetów powyżej pochodzi z innej procedury oceny i nie jest wynikiem take7.

8. Aplikacja

Dokładność rozpoznawania nie jest równoznaczna z użytecznością trenera. Trzy zagadnienia okazały się mieć wagę porównywalną ze zmianami w modelu.

8.1. Wymóg pełnego okna kontekstowego

Trener wyznaczał okna **wyłącznie wewnątrz** wybrzmiewającego akordu (`range(start, koniec - 48)`). Po uderzeniu w struny bufor aplikacji przez 0,77 s zawiera częściowo ciszę; uwzględniając okno FFT (8192 próbki, czyli 512 ms), najstarsza ramka opisuje sygnał sprzed nawet 1,3 s. Stanowi to wejście spoza rozkładu treningowego.

Zaobserwowany objaw: akordy septymowe rozpoznawane były dopiero w fazie wybrzmiewania, to jest w pierwszym momencie, w którym okno zostaje w całości wypełnione akordem.

Zastosowanym rozwiązaniem był początkowo jeden próg: aplikacja nie kierowała zapytania do modelu, dopóki okno nie było wypełnione sygnałem w 90%. Próg ten jest właściwy dla NAZWY akordu i niewłaściwy dla wszystkiego pozostałego. Granie po jednym dźwięku nigdy nie wypełnia okna w dziewięciu dziesiątych — to 43 ramki, czyli 688 ms nieprzerwanego dźwięku — wobec czego model nie był pytany wcale, ekran zastygał na ostatnim akordzie, a wraz z nim zastygało wszystko, co z niego korzysta.

Wymaganie jest obecnie rozdzielone na dwa. Model pytany jest od połowy okna (pomiar: przy wypełnieniu 50–70% głowica wysokości nazywa odosobniony dźwięk poprawnie w każdej ramce pomiaru), a jego nazwie akordu wierzy się dopiero od dziewięciu dziesiątych. Oba progi zapisane są jako stałe w jednym miejscu aplikacji, ponieważ bramka na nazwie stosowana jest po drugiej stronie kanału niż wątek pytający — zapisane dwukrotnie mogłyby się rozjechać.

8.2. Zróżnicowany czas wybrzmiewania składników akordu

Podgląd diagnostyczny przytrzymanego akordu Gm7:

G m7	min7=96%	b7=96	← bezpośrednio po uderzeniu
G m7	min7=82%	b7=76	
G m7	min7=52%	b7=52	
G m	min=49%	b7=45	← septyma wyciszona, model zmienia klasyfikację

Klasyfikacja modelu jest poprawna — w bieżącym oknie septyma faktycznie nie występuje. Akord nie zmienia jednak tożsamości w trakcie wybrzmiewania.

Zastosowane rozwiązanie: zatrask jakości. Mechanizm **załącza się** przy pewności $\geq 0,60$, natomiast **utrzymuje stan niezależnie od niej**. W fazie zaniku model raportuje uboższą jakość z pewnością rzędu 94–96%, wobec czego sam próg pewności nie stanowiłby zabezpieczenia. Zwolnienie zatrasku następuje przy nowym ataku lub przy zmianie prymy.

Zatrask łączy się dopiero po 48 klatkach od ataku. Wcześniej okno zawiera jeszcze ogon **poprzedniego** akordu, co skutkowałoby zatrzaśnięciem nieprawidłowej nazwy.

Detekcja ataku porównuje poziom sygnału z wolnozmienną obwiednią (EMA), a nie z progiem bezwzględnym, który zależałby od głośności wykonania. Zastosowano ponadto refrakcję 0,2 s, aby pojedyncze szarpnięcia wyzwalало dokładnie jeden atak.

8.3. Pomiar czasu zamiast wartości założonej

Licznik postępu otrzymywał wartość stałą $dt = 0,040$, podczas gdy wątek inferencji wymagał 55–90 ms na cykl (inferencja wraz z 40 ms uśpienia). Licznik pracował zatem wolniej od zegara rzeczywistego: próg 0,6 s osiągany był po około sekundzie, przy czym wartość zależała od obciążenia maszyny.

Po korekcie, obejmującej pomiar czasu rzeczywistego, wyznaczanie okresu przez wątek inferencji od początku cyklu, zawężenie okna głosowania z 5 do 3 oraz obniżenie domyślnego progu do 0,25 s, przejście trwa **około 0,3 s** zamiast 1,2 s.

8.4. Rzadka reprezentacja jądra CQT

Pełne jądro obejmuje $4097 \times 144 = 589\,968$ wag, skoncentrowanych wokół częstotliwości środkowej każdego binu. Odrzucenie wag poniżej $1e-4$ wartości szczytowej daje następujące wyniki:

próg	zachowanych wag	maks. błąd względem szczytu
1e-5	21,9%	0,006%
1e-4	6,9%	0,033%
1e-3	2,3%	0,352%

Błąd wyznaczono na trzech widmach: białym, różowym oraz harmonicznym serii gitarowej. Po transformacji CQT następuje log-normalizacja do zakresu 80 dB, wobec czego wartość 0,03% pozostaje o rzędy wielkości poniżej rozdzielczości cechy.

Rozmiar pliku wag zmniejsza się z **28 MB do 2 MB**, a ścieżka audio wykonuje około **14-krotnie mniej mnożeń** na ramkę. Poprzednia implementacja przechodziła wszystkie 4097 binów FFT dla każdego ze 144 binów CQT, odsiewając wartości zerowe dopiero wewnątrz pętli.

8.5. Zgodność cech pomiędzy trenerem a aplikacją

Rozbieżności tej klasy są szczególnie trudne w diagnozie, ponieważ aplikacja pozostaje funkcjonalna, wykazując jedynie błędy klasyfikacji. Zidentyfikowano dwie:

- **mapowanie chromy.** Plik dystrybuowany z aplikacją związał biny parami $(0,1)$, $(2,3)$, ..., natomiast `librosa.cq_to_chroma` stosuje podział $(1,2)$, $(3,4)$, Co drugi bin trafiał do klasy sąsiedniej, co odpowiada rozmyciu chromy o pół tonu na połowie pasma.
- **klucz pamięci podręcznej.** Nazwa pliku cache pochodziła z wyrażenia `abs(hash(ścieżka))`. Interpreter Pythona losuje ziarno funkcji skrótu dla łańcuchów przy każdym uruchomieniu procesu, wobec czego pamięć podręczna nie była wykorzystywana pomiędzy sesjami. Obecnie stosowany jest skrót SHA-1 z nazwy pliku.

Aplikacja **odrzuca** wagi w poprzednim, gęstym formacie, sygnalizując to komunikatem.

8.6. Interfejs użytkownika

Po przeglądzie liczba regulatorów została zredukowana z pięciu do czterech:

regulator	uwagi
Bramka szumu	w dBFS, z miernikiem poziomu w tej samej skali i znacznikiem progu
Pewność akordu	próg dla nazwy akordu (tryb Akordy)

regulator	uwagi
Próg dźwięku	próg dla pojedynczego dźwięku (tryby dźwiękowe)
Czas przytrzymania	wymagany czas utrzymania poprawnego akordu

Usunięto regulatory `Tail` (ustawiany z interfejsu i nieodczytywany w żadnym miejscu kodu) oraz `In gain` (którego wpływ znosiła normalizacja wykonywana w obrębie ramki, wobec czego przesuwiał on wyłącznie tę samą nierówność co bramka szumu).

Regulator `Confidence` sterował dwiema różnymi wielkościami jednocześnie, a w trybach dźwiękowych podlegał ograniczeniu dolnemu `.max(0.5)`, w wyniku czego cały zakres 0,1–0,5 dawał identyczne zachowanie. Funkcje rozdzielono.

Bramka szumu operowała uprzednio w liniowej skali RMS 0–0,1, która **nie obejmowała poziomu szumu mikrofonu laptopowego** (RMS 0,05–0,15 po wzmocnieniu). Skala decybelowa –72... 0 dBFS zapewnia rozdzielczość w wymaganym zakresie oraz zasięg do pełnej skali.

Panel przerósł od tego czasu pojemność jednej kolumny i podzielony jest na cztery zakładki — wejście wraz z bramką, surowość oceny, materiał do zagrania oraz zawartość okna. Trzecia z nich zawiera wyłącznie to, co należy do trybu widocznego na ekranie: utwór nie ma nic do powiedzenia w Formułach, a formuła nic w Akordach, więc w każdym trybie jest to inna zakładka. Rysowana jest zawsze jedna zakładka, wobec czego odświeżaniu w trakcie gry podlega odpowiednio mniej.

8.7. Tryb diagnostyczny

```
SOLITITO_DEBUG=1 ./solitito
```

Tryb wypisuje przy każdej predykcji trzy najsilniejsze jakości oraz wektor pitch przeliczony na **interwały względem rozpoznanej prymy**:

```
G m7 | min7=97% sus=0% maj=0% | R96# b25 28 b382# 37 44 b56 594# b616 69 b797# 74
```

Narzędzie rozróżnia przypadek, w którym model nie wykrywa septymy, od przypadku, w którym ją wykrywa, lecz pomija w klasyfikacji. Oba objawy są nierozróżnialne na poziomie nazwy akordu i prowadzą do przeciwnych działań korygujących. Przypadek `Gm7` rozstrzygnięto przy jego użyciu bez ponownego treningu.

8.8. Pojedynczy dźwięk nie jest pytaniem, na które model potrafi odpowiedzieć

Rozdział opisuje wcześniejsze rozwiązanie CQT i jego pomiary. `Take7` zachowuje ten tor, ale ćwiczenie wymagające świeżego uderzenia korzysta z niezależnych zdarzeń Rise opisanych w 8.12, zamiast czekać na okno akordowe.

Model pytany jest o 48 ramek — 0,77 s — i odpowiada o całości tego materiału. Jest to właściwe dla akordu trzymanego pod palcami i niewłaściwe dla gamy, w której dźwięki następują po sobie szybciej, niż okno zdąży się opróżnić.

Pomiar narzędziem `--probe` na gamie granej po 0,6 s na dźwięk, przy obowiązującej wówczas regule (klasa docelowa powyżej progu i w granicach 10% od najgłośniejszej):

	dźwięk aktualnie grany	dźwięk poprzedni
głowica wysokości modelu	7% okien	79%

	dźwięk aktualnie grany	dźwięk poprzedni
pojedyncza ramka CQT	57%	43%

Wina nie leży po stronie modelu: na dźwiękach izolowanych i trzymanyh przypisuje on 0,96–0,99 właściwej klasie, a na gamie raportuje oba dźwięki, ponieważ oba znajdowały się w oknie. Starszy z nich wygrywa poziomem, mając za sobą większą część okna.

Przyjęte rozwiązanie: tryby nutowe zadają drugie pytanie pojedynczej ramce CQT, pozbawionej pamięci. Suma harmoniczna po logarytmicznych prążkach — wobec logarytmicznej osi jest to widmo iloczynu harmonicznym — wskazuje klasę wysokości brzmiącą w danej chwili. Ani razu nie wskazała klasy, która nie została zagrana.

Domyślnie oszacowanie to wyłącznie **dokładą** drogę do zaliczenia, ponieważ oddanie mu rozstrzygnięcia kosztowałoby własność odróżniającą ten trenażer od monofonicznego: głowica wysokości jest polifoniczna, więc akord zagrany jednym pociągnięciem zalicza swoje interwały po kolei. Opcja **Graj dźwięki pojedynczo** czyni oszacowanie rozstrzygającym i dodatkowo wymaga nowego ataku, zanim powtórzony dźwięk zostanie zaliczony po raz drugi.

Pozostałe opóźnienie wnosi okno FFT o długości 8192 próbek, czyli pół sekundy, i to ono sprawia, że dźwięki krótsze niż około 0,4 s pozostają trudne. Estymator o krótszym oknie, działający w dziedzinie czasu (autokorelacja), jest drogą, która pozostaje otwarta.

8.9. Wybór wejścia i to, czego lista urządzeń nie pokazuje

Maszyna z systemem Windows nie podawała sygnału do momentu ręcznej zmiany częstotliwości próbkowania, co wykazało, że format próbki zwracany przez backend nie może być pomijany, a wybór urządzenia należy do użytkownika, nie do domyślnej konfiguracji systemu.

Lista urządzeń ma jedną własność nieoczywistą: **kartę można otworzyć raz**. Cokolwiek ją trzyma — serwer dźwięku, inna aplikacja albo własny strumień tego programu — usuwa ją z wyliczenia całkowicie. Wynikają z tego trzy konsekwencje, z których każda została najpierw zaobserwowana jako usterka:

- lista zbudowana po otwarciu strumienia nie zawiera karty, z której trwa nagrywanie,
- pod PipeWire, który przejmuje sprzęt, pozostają wyłącznie cztery nazwy serwerowe,
- urządzenie, z którego nagrywamy, musi być wyłączone spod oznaczenia „niedostępne”, ponieważ jego nieobecność w skanie jest właśnie dowodem, że działa.

Bramka szumu zapamiętywana jest per urządzenie. Interfejs i mikrofon laptopa dzielą dziesiątki decybeli, a próg, który trzeba odnajdywać po każdym przełączeniu, nie jest ustawieniem.

8.10. Niezależne harmonogramy inferencji

Inferencja akordów działa co 40 ms, a Rise przetwarza każdy skok audio 16 ms w osobnym wątku. `--bench` mierzy gałąź akordową, nie cały tor onsetów i zaliczania. Wydzielenie gałęzi połączonego ONNX w pamięci podczas ładowania pozwala nie uruchamiać enkodera akordowego przy analizie samych onsetów.

Wcześniejsze pomiary 39 ms na Linuksie i 61 ms na Windows dotyczyły inferencji akordowej na maszynie odniesienia. Są to pomiary historyczne, a nie opóźnienie gałęzi Rise w `take7`. Obciążenie CPU należy również porównywać na tej samej podstawie: jeden w pełni zajęty rdzeń odpowiada 12,5%, gdy licznik obejmuje osiem rdzeni.

8.11. Formuły oraz reguła surowsza niż w trybach dźwiękowych

Poniższe porównanie dokumentuje wcześniejszy tor CQT/take6. W obecnej wersji włączenie **Zaliczaj tylko to, co uderzone** kieruje również Formuły do Rise; po wyłączeniu opcji obowiązują dotychczasowe reguły brzmiących nut.

Aplikacja losuje zbiór interwałów nad prymą — każdy podzbiór dwunastu funkcji chromatycznych zawierający prymę, łącznie 2048 — a ćwiczenie polega na odnalezieniu ich na gryfie i zagraniu w dowolnej kolejności. Funkcja raz zaliczona pozostaje zaliczona do końca rundy, co zmienia koszt zaliczenia fałszywego: w pozostałych trybach błędny odczyt opóźnia ćwiczenie, tutaj usuwa z niego funkcję bezpowrotnie.

Regułę zatem zmierzono, zamiast ją założyć. Na 49 dźwiękach rzeczywistego nagrania (`dist/latency_stats.py`):

reguła	zaliczenia fałszywe	dźwięki pominięte
cztery ścieżki naraz, jak w trybach dźwiękowych	110	0
sama jednoramkowa estymata CQT	33	0
to samo, bramkowane głowicą ataków	15	4

Ze 110 dziewięćdziesiąt dziewięć pochodziło z głowicy wysokości modelu — która odpowiada na pytanie „co brzmi”, a struna wybrzmiewająca albo rezonująca współczująco brzmi, nie będąc zagrana. Tamta wersja korzystała więc z samej estymaty jednoramkowej, z głosowaniem czterech z pięciu ostatnich ramek audio. Starej bramki atakowej nie przyjęto, ponieważ w tym porównaniu pomijała cztery zagrane dźwięki.

Ten sam tryb potrafi również postawić formułę na akordzie: jej pryma sadzana jest na jednym z dwunastu stopni akordu, po czym zliczane jest, ile z tego akordu zbiór pokrywa — wszystkie jego dźwięki poza prymą i czystą kwintą, bo tylko one ustalają, jaki to akord. Jest to arytmetyka na dwóch dwunastobitowych maskach i jest dokładna, co czyni ją wartą pokazania na ekranie obok funkcji.

8.12. Zdarzenia i granice ćwiczeń

Brzmiąca wysokość i świeży atak dostarczają innego rodzaju dowodów. Przy Rise i włączonym **Zaliczaj tylko to, co uderzone** aplikacja zużywa zdarzenia opisane klasą wysokości, czasem audio, identyfikatorem ramki i generacją wejścia. Zdarzenia już zużyte, wygasłe lub pochodzące sprzed granicy nie mogą zaliczyć nowego celu. Pauza, zmiana ćwiczenia i restart wejścia czyszczą oczekujące zdarzenia; przerwa kolejki audio resetuje kontekst detektora, zamiast udawać ciągły strumień.

Atak wielodźwiękowy może dostarczyć kilka zdarzeń klas wysokości dla jednego ćwiczenia. Reguły kolejności i grania pojedynczo określają, które z nich mogą zostać użyte. Ponowne uzbrajanie detektora względem szczytu prawdopodobieństwa jest zachowywane przez zmiany ćwiczeń; sama zmiana celu nie jest nowym atakiem.

Opcjonalne `SOLITITO_ONSET_RESCUE=1` potwierdza słabszą odpowiedź przyrostem widma danej wysokości i dodatkową ramką. Znacznik czasu pozostaje czasem kandydata, więc potwierdzenie nie przenosi wcześniejszego ataku za granicę rundy. Silne odpowiedzi nie wymagają tego dodatkowego oczekiwania. Rescue jest domyślnie wyłączony i niezależne od wyboru modelu oraz nagrywania audio.

Zwykły start automatycznie wybiera `best_model_v2_take7.onnx`, jeśli plik jest dostępny. `SOLITITO_MODEL` nadpisuje główną ścieżkę, a `SOLITITO_ONSET_MODEL` źródło onsetów. Do porównania nadal dostępne jest take6 z `SOLITITO_ONSET=legacy`. Skrypt logowania/nagrywania jest opcjonalny; zwykle ćwiczenie nie wymaga logu ani zapisu WAV. Zobacz [uruchamianie aplikacji](#).

9. Hipotezy zweryfikowane negatywnie

Wpisy o głowicy onsetów w tej tabeli dotyczą wcześniejszego toru take6/CQT, nie obecnej gałęzi Rise.

Rozdział dokumentuje przypadki, w których pomiar obalił wcześniej przyjęte założenie.

hipoteza	wynik pomiaru
Rozbieżność normalizacji (w ramce wobec globalnej) stanowi główne ograniczenie	różnica poniżej 1 pp; warstwa <code>InstanceNorm2d</code> ją kompensuje
Bariera harmoniczna — trzecia harmoniczna b3 przypada na b7, wobec czego min7 i min są nierozróżnialne	pomiar wykonano na segmentach z etykietami losowymi
Model nie odwzorowuje danych treningowych (niedouczenie)	kontrola TRAIN: 83,7% wobec 63,1% na walidacji, a więc przeuczenie
Dopasowanie szablonów przewyższy głowicę quality ($B \approx 75\%$ wobec $A = 63\%$)	$B = 61,0\%$
Sufit prymy na poziomie 64% wynika z voicingów jazzowych bez prymy	artefakt nagrań solowych; w akompaniamencie 97%
Maskowanie prymy odblokuje jakość powyżej 73%	jakość pozostała na poziomie 72%
Chroma w dystrybuowanym pliku jest jednoelementowa, a więc nieprawidłowa	<code>cq_to_chroma</code> przy 24 binach na oktawę również przypisuje jedną wagę na bin; rozbieżność dotyczyła przesunięcia
Bez zmiennej <code>ORT_DYLIB_PATH</code> binarka wykorzysta bibliotekę systemową	<code>RUNPATH=\$ORIGIN</code> z pliku <code>.cargo/config.toml</code> rozwiązywał to zagadnienie
Model pogorszył się w rozpoznawaniu pojedynczych dźwięków	na dźwiękach izolowanych przypisuje 0,96–0,99 właściwej klasie; na gamie jego okno 0,77 s zalicza dźwięk poprzedzający grany, w 79% okien
Akord zmniejszony septymowy nazwany od innego swojego dźwięku to inny akord	to te same cztery dźwięki: <code>Cdim7</code> , <code>Ebdim7</code> , <code>Gbdim7</code> i <code>Adim7</code> różnią się wyłącznie tym, który z nich model uzna za prymę, a to wynika z układu chwytu, nie z tego, co zagrano
Głowica ataków będzie lepszą bramką — jest najszybszą dostępną odpowiedzią	na nagraniu tak wyglądało: 202 ms wobec 676 ms. Zastosowana na żywo odrzucała znacznie więcej, niż wyłapywała, a w regule zaliczania wymieniła 18 zaliczeń fałszywych na 4 dźwięki pominęte
Tercja zaliczona przy granej prymie to piąta harmoniczna tej prymy	<code>--probe</code> na 364 oknach: zaliczenia fałszywe padają na +10 i +11 półtonów, czyli na dźwięk POPRZEDNI, wciąż obecny w oknie, a nie na harmoniczną
Jedna świeża odpowiedź głowicy ataków to okno zbyt krótkie, by uchwycić uderzenie	przy progu 0,02 odpowiedź utrzymuje się nad progiem przez medianę jednej sekundy po ataku, a żaden z 47 dźwięków nie został bez ramki, która by ją niosła — trzymaną w tym celu pamięć szesnastu ramek usunięto

Zależność jest jednoznaczna: **wyniki pomiarów potwierdzały się konsekwentnie, natomiast przewidywania formułowane przed pomiarem okazywały się błędne w sposób systematyczny.** Uzasadnia to przyjętą metodykę opartą na sondach.

10. Decyzje projektowe

10.1. Rozwiązania przyjęte

Generator wypisuje etykiety wprost. Żaden krok przetwarzania nie odtwarza informacji z sygnału.

Weryfikacja etykiet przez niezależny skrypt. Rozwiązanie nadmiarowe względem generatora, zastosowane celowo.

Podział zbioru po źródle. Obniża raportowane wskaźniki o kilkanaście punktów procentowych i jest uzasadniony.

Cztery wyjścia, dwie niezależne gałęzie. Root, quality i pitch zachowują bazę akordową. Rise dostarcza ataków ze znacznikami czasu do ćwiczeń wymagających świeżego uderzenia. Obie gałęzie są dystrybuowane w jednym ONNX, z osobnymi wejściami i harmonogramami inferencji.

Dwa progi na oknie kontekstowym zamiast jednego. Model pytany jest od połowy okna, a jego nazwie akordu wierzy się od dziewięciu dziesiątych — jeden próg nie może obsłużyć zarazem trzymanego akordu i pojedynczego dźwięku.

Nic, co usłyszano przed granicą, jej nie przekracza. Zmiana akordu, koniec rundy i przełączenie trybu porzucają ostatnią odpowiedź modelu, ponieważ dotyczy ona tego, co było przed nimi.

Rzadka reprezentacja jądra CQT. Korzyść dwojaka: rozmiar pliku wag oraz czas przetwarzania w wątku audio.

Odrzucanie niezgodnych wag przez aplikację. Cicha akceptacja skutkowałaby programem funkcjonalnym, lecz błędnie klasyfikującym.

Napisy wkompileowane, bez biblioteki gettext. Przy kilkudziesięciu napisach zależność systemowa oraz katalogi `.mo` generują koszt przewyższający korzyść.

10.2. Rozwiązania odrzucone

Wyprowadzanie jakości z wektora pitch — zmierzone jako gorsze o 21 punktów procentowych od głowicy quality.

Agregacja czasowa jako sposób poprawy jakości — na kontrolowanej populacji daje około +1 pp. Błędy modelu są skorelowane w czasie: model nie wykazuje wahania pomiędzy oknami, lecz konsekwentnie i z wysoką pewnością wskazuje tę samą nieprawidłową odpowiedź.

Tryb taktowy (zapis przesuwający się w tempie, ocena zamiast bramki) — zaimplementowany, a następnie wycofany. Przyjęto założenie, że trener ma reagować dynamicznie.

Faza 3 treningu — zmierzona jako nieprzynosząca poprawy w trzech przebiegach.

10.3. Dalsza weryfikacja

- **Niezależne nagrania z docelowych instrumentów i interfejsów.** AtoA i wcześniej przeanalizowane nagrania z ćwiczeń są materiałem regresyjnym; kontrolowane porównanie wag wymaga dodatkowego nieznanego materiału.

- **Stabilne wznowianie i pochodzenie danych.** Wybrany model, próg, tożsamości danych i raport treningu należy zachowywać razem. Kontroli zmienionych danych nie należy obchodzić, aby wznowić niepowiązany przebieg.
 - **Pakowanie wydania.** Workflow 0.5.7 dołącza połączony artefakt take7, sprawdza jego sumę SHA-256 i wykonuje obie gałęzie modelu przez `--check` w przygotowanych paczkach Linuksa i Windows przed publikacją.
-

11. Podsumowanie

W pracach nad take6 uzyskano model osiągający 92,4% trafień dokładnych na walidacji wyznaczonej z podziałem po źródle, wydany jako pakiety dystrybucyjne dla dwóch platform.

Zasadniczy przyrost dokładności nie wynikał ze zmian architektury, lecz z czterech ustaleń dotyczących danych:

1. połowa zbioru GuitarSet stanowi improwizację opisaną akordami akompaniamentu,
2. adnotacja `instructed` nie zawiera septym i błędnie klasyfikuje pięćset segmentów,
3. podział zbioru na poziomie segmentów wprowadza przeciek,
4. cele pitch należy wyznaczać z rzeczywistego wykonania, nie z zapisu.

Wymienione cztery zmiany przesunęły wskaźnik `Exact` z 44,8% na 92,4%. Żadna z nich nie dotyczyła struktury sieci.

Take7 wprowadza kolejny etap rozwoju: krótkie przyczynowe cechy onsetów, sieć Rise wyspecjalizowaną w świeżych atakach oraz zdarzenia ze znacznikami czasu w aplikacji. Baza akordowa może zostać wykorzystana bez ponownego treningu, a cały model można też wytrenować od zera. Zweryfikowany eksport dostarcza obie gałęzie w jednym ONNX, zachowując ich niezależne harmonogramy pracy. Rozszerza to aplikację z rozpoznawania brzmiącego materiału o ocenę nowo nagranych dźwięków.

Aktualizacja: 4 października 2026 — take7, aplikacja 0.5.7. Repozytorium: <https://github.com/greblus/solitito>